

Whole-year text: define the contract first

7 October 2026 | C003a grammar and range | Synthetic teaching examples

Sidiq review status: Not yet reviewed.

Research question and hypothesis

What text should count as a valid whole number of years? Hypothesis: successful conversion to an integer is insufficient. We also need an explicit text grammar, complete-input validation and a representable nonnegative result. Today's one concept is defining that input contract before code can silently change the meaning of a request.

October 5 showed that converting 2.5 to int discards the fraction before C002 can validate it. October 6's run failed before saving its teaching material; it left no pending note or PDF. Today finishes that prerequisite with its true work date. The parser function moves to the next session; we do not compress two lessons to catch up. C003a remains in progress and C003b is deferred.

Beginner explanation and terms

Text is a sequence of characters. Grammar specifies which sequences are allowed. Parsing interprets permitted text as a value. Range is the set of values the receiving type can represent. Full consumption means every input character has been checked, including any suffix or invisible byte. These are separate questions: could an integer be read, was all input used, and does it meet our model?

Our whole-year count is $n = 0, 1, 2, \dots$ up to the largest representable int. It is measured in whole model years. It is neither a date nor an annual interest rate. A blank entry does not mean zero. The model allows zero years; an explicit zero carries information that missing input does not.

The chosen contract

Accept one or more ASCII digits 0 through 9, interpreted in decimal, with value at most `std::numeric_limits<int>::max()`. Allow leading zeros. Reject signs, whitespace anywhere, decimal points, exponent notation, separators, suffixes, embedded NUL bytes and non-ASCII digit encodings. No trimming, rounding or fractional truncation is allowed. Every byte in the supplied range counts.

These are project choices. No normalization keeps the first parser explicit. There is no additional text-length limit; future external-input protection must address memory and transport limits.

Example text	Hand decision	Why
0	Accept as 0	Zero is a valid whole count
002	Accept as 2	Leading zeros, decimal interpretation
Empty text	Reject	No number supplied
2.5 or 2.0	Reject	A decimal point is outside the grammar
-1, -0 or +2	Reject	All signs are forbidden
Space then 2, or 2 then space	Reject	No whitespace trimming
2x or 2e0	Reject	Unconsumed or non-digit text

The table records requirements. It is not evidence that a whole-year parser has passed these cases.

Mathematics and a worked example

For digits d_1 through d_k , start $n_0 = 0$. Append each digit with $n_j = 10 * n_{j-1} + d_j$. For 002 the successive values are 0, 0, 2. Leading zeros therefore change the spelling, not the value. The recurrence describes exact mathematical meaning. A C++ implementation must check bounds before an operation can overflow, or use a checked library conversion. Overflow is not a valid rejection test.

The local type probe reports int maximum $M = 2147483647$. The decimal string for M is allowed; the string for $M + 1 = 2147483648$ is not representable here. Python integer arithmetic checks the successor without overflowing a C++ int. The contract queries M on each compilation target and does not assume every target has the same int width. A string of many zeros still represents zero.

If accepted text 002 supplies $n = 2$, the existing financial model is $B = P * (1 + r)^n$. With $P = 1000$ currency units and effective annual rate $r = 0.05$, year one adds 50 to reach 1050. Year two adds 52.50 to reach 1102.50. Decimal arithmetic independently reproduces these two interest credits. Text 2.5 cannot be silently substituted into that two-year calculation.

A small C++ library experiment

The fixed conversion_contract.cpp example observes two calls to `std::from_chars`. It does not expose external input or implement the contract. Before running, the expected output specifies these facts:

Fixed input	Conversion succeeds	Uses all input	Returned integer
2x	true	false	2
-1	true	true	-1

The first case proves why an error-code check alone misses trailing text. The second proves why complete conversion alone does not impose our nonnegative digit-only policy. Neither should reach the savings function. In particular, checking only a nonnegative result would not enforce the ban on a signed zero spelling; the original characters matter.

Introduce the syntax before reading the source

The `charconv` header supplies conversion, `string` supplies text, `iostream` supplies printing, and `system_error` supplies error codes. `std::string` holds the original characters. `const` keeps each fixed string unchanged. `int suffix_value = 0` initializes the receiving integer. `auto` lets the compiler infer the result type from the expression on the right.

Read the first block in order: store 2x; initialize a receiving integer; call `from_chars` on the range from `data()` to `data() + size()`; print the error comparison, stopping-position comparison and integer. `data()` identifies the first character and `size()` counts the input characters. The end position is just after the last character. The library writes through its integer reference parameter and returns a result containing an error code `ec` and stopping position `ptr`.

The expression `ec == std::errc{}` asks whether conversion succeeded; `ptr == end` asks whether it used the entire range. Equality produces a Boolean, and `boolalpha` prints true or false. The second block repeats the same sequence for -1. The final line states the contract consequence. The project worksheet explains this source in order; the next session introduces parser branches.

Evaluation protocol, implementation and observed results

Inputs are synthetic. Predeclared checks are exact output agreement, repeatability, the local type boundary and an independent balance. The hand table remains a future parser test plan.

Added `years_contract.md`, `conversion_contract.cpp` and `conversion_contract_expected.txt` to the savings project. Make adds `cpp-conversion-contract` and includes its output comparison in `cpp-check`. C002's savings function, historical outputs and earlier research evidence remain unchanged.

Check	Observed result
New C++ demonstration	Both library observations match the hand-authored output
Two independent executable runs	Both match the complete expected bytes
Range probe	Local int maximum is 2147483647
Independent Python arithmetic	Maximum successor 2147483648; 002 means 2; balance 1102.50
Full suite	608 Python tests, 66 existing C++ checks and three output fixtures pass
Quality checks	Ruff lint, 49 formatted files and strict mypy on 21 source files pass
Build and dependencies	Fresh warning-clean C++17 build; pip consistency passes

Run `make cpp-conversion-contract`, `make check` and `make -B cpp-check`. Compilation uses Wall, Wextra, Wpedantic and Werror. Scratch evidence is in `work/2026-10-07`. No new dependency, parser-test count, coverage or speed claim. Implementation checks passed first run. Render with `make note NOTE_DATE=2026-10-07`.

Assumptions, costs and failure cases

Savings assumes a constant effective annual rate, reinvestment and no cash flows, fees, tax or inflation. Costs reduce wealth; none is estimated. Parsing cannot validate economic assumptions. There are no trades, empirical returns or investment recommendations here.

An accepted count can still overflow the positive-rate savings calculation; C002 retains separate overflow checks. Rejecting 2.0 is deliberate syntax policy. Leading zeros do not imply octal. Embedded NUL rejection requires an interface that preserves input length rather than truncating.

The probe is not a parser, portability proof or security audit. The future function must reject invalid input without a usable year value; its error API is next session. Tests do not prove mastery.

Primary sources consulted on 7 October

Microsoft's `from_chars` reference explains error codes, the stop position, representability errors and non-skipped whitespace. Our grammar adds explicit project policy to those library mechanics.
<https://learn.microsoft.com/en-us/cpp/standard-library/charconv-functions?view=msvc-170>

Microsoft's `numeric_limits` reference explains how to query the type's largest finite value. The actual local int bound was observed with a compiled probe; it was not inferred from a platform name.
<https://learn.microsoft.com/en-us/cpp/standard-library/numeric-limits-class?view=msvc-170>

Interview explanation

I would separate spelling, range and model meaning. A successful conversion can consume just a prefix, and a fully consumed signed integer can still violate a whole-year text policy. I would preserve the original text, define the permitted grammar, check the whole range and reject values that cannot fit. Only an accepted integer would reach the savings function, which still checks its own financial inputs and numerical result. Today establishes that contract, not the final parser.

Progress, cadence and next task

The October 6 grammar prerequisite is completed on October 7 with this note and library experiment. The failed prior run has no fabricated deliverable or review entry. Next is one parser function; then independent boundary verification, the fractional-year failure example, reproduction and consolidation remain in order. If needed these displace later dates rather than adding extra lessons. C003b, vectors/statistics and the ledger remain conditional on readiness.

While holding the shared run lock, today's persisted London-date draw was 2. Actual history and both run memories showed zero commits today before work. One coherent validated lesson is ready; we plan one commit, leaving the second slot unused rather than splitting this evidence. Final commit, push verification and actual count are reported in the run memory and daily briefing.

No input is needed for the next synthetic lesson. Q004a lawful entitlement/sample and Q005b integration gates remain prerequisites to empirical work. Publication authority is separate from Sidiq's personal review, which remains Not yet reviewed. The daily schedule remains 08:00 Europe/London including daylight-saving changes. Stop new research after 31 December 2026 and request the next programme scope. No schedule change was needed.

Check your understanding

1. Why does a successful conversion of 2x not satisfy our contract?
2. Why is a completely converted -1 still rejected? What about the spelling -0?
3. Which of 002, empty text, 2.0 and a space followed by 2 is accepted, and with what value?
4. What are the two distinct ways a large year count can fail, before and after parsing?
5. Work through 002 as decimal digits, then two annual interest credits from 1000 at 5%.

Suggested answers

1. Conversion stopped at x. The numeric prefix was readable, but the full input was not accepted. The stop position must be checked, alongside the error result and the grammar.
2. The minus sign violates the digit-only rule and -1 is outside the nonnegative model. The -0 spelling is also forbidden even if a numeric conversion would produce zero. Inspect the text.
3. Only 002 is accepted, as 2. Empty text has no digits; 2.0 contains a forbidden point; the leading space is rejected because this contract does not trim whitespace.
4. Text above the target int maximum cannot become a valid receiving integer. A representable count can nevertheless cause the savings calculation to overflow at a positive interest rate. These are separate validation boundaries, and a passing parser does not guarantee a valid balance.
5. Starting at zero, the appended digits yield 0, 0 and 2. Interest is 50 then 52.50, giving 1050 then 1102.50. These are synthetic model years under fixed-rate, no-cost assumptions.